



南開大學
Nankai University

计算机学院

NTT 选题：SIMD 优化

姓名：钱展飞

学号：2312479

专业：计算机科学与技术

2026 年 5 月 8 日

目录

1 概述	2
2 NTT 算法原理	2
2.1 FFT 算法原理	2
2.1.1 FFT 概述	2
2.1.2 FFT 原理	2
2.2 NTT 原理	3
2.3 蝴蝶变换	3
3 SIMD 优化	3
3.1 NEON Intrinsics	3
3.2 Montgomery 乘法	4
3.2.1 Montgomery 乘法的底层原理：在取模乘法下对 2 的 n 次幂进行快速除法 . . .	4
3.2.2 Montgomery 形式下的运算	4
3.3 使用 SIMD 优化	5
3.3.1 对于向量的选择	5
3.3.2 使用 SIMD 实现 Montgomery 形式下的运算	5
3.3.3 使用 SIMD 改进 NTT	7
3.3.4 使用 SIMD 后的理论复杂度分析	8
3.4 使用 SIMD 优化后的性能分析	8
4 DIF & DIT	8
5 总结	9

1 概述

多项式乘法是科学计算中十分基础且核心的数学运算形式，广泛应用于数字信号处理、计算机图形渲染、密码学安全算法等多个工程与学术领域。本次并行程序设计实验以 NTT 多项式乘法为研究对象，重点面向同态加密场景下的多项式运算展开优化研究。同态加密技术支持直接在密文数据上执行各类算术运算，解密后所得结果与明文直接运算结果完全一致，是隐私计算、安全云计算领域的关键支撑技术，而多项式乘法正是同态加密方案中依赖度极高的底层核心模块。

2 NTT 算法原理

NTT 数论变换算法建立在 FFT 快速傅里叶变换的理论框架之上，因此本章先梳理 FFT 算法的基础思想与推导逻辑，再阐述从复数域 FFT 向模整数域 NTT 演进的内在原因，最后介绍迭代版 NTT 必需的蝴蝶变换机制。

2.1 FFT 算法原理

2.1.1 FFT 概述

对两个多项式 $F(x), G(x)$ 做乘法时，若采用朴素逐项相乘的实现方式，整体时间复杂度高达 $O(n^2)$ 。

若将多项式由系数表示转换为点值表示形式，多项式相乘仅需将对应位置点值逐一相乘，时间复杂度可降至 $O(n)$ 。依据代数基本定理相关推论， n 次多项式可由 $n+1$ 个互异采样点唯一确定，再通过插值算法还原为系数形式。传统拉格朗日插值复杂度为 $O(n^3)$ ，即便通过选取特殊采样点优化至 $O(n^2)$ ，依旧无法满足大规模多项式乘法的效率需求。

FFT 算法通过引入复平面单位根作为采样节点，能够在 $O(n \log n)$ 复杂度下完成多项式系数与点值表示的双向快速转换，最终将多项式乘法整体复杂度优化至 $O(n \log n)$ 。

2.1.2 FFT 原理

定义 n 阶复数单位根 $W_n = e^{\frac{2\pi i}{n}}$ ，序列 W_n^k 对应复平面单位圆上均匀分布的 n 个等分点。

对于长度为 n 的离散序列 x ，其离散傅里叶变换第 k 项定义为：

$$DFT(x, n)_k = \sum_{i=0}^{n-1} x_i W_n^{ki}$$

将多项式系数视作离散序列后，该序列的 DFT 结果等价于多项式在各阶单位根处的点值采样。直接暴力求解 DFT 复杂度仍为 $O(n^2)$ ，但利用单位根的周期性与对称性可实现分治拆解：

$$\begin{aligned} DFT(x, n)_k &= \sum_{i=0}^{n-1} x_i W_n^{ki} \\ &= \sum_{i=0}^{\frac{n}{2}-1} x_{2i} W_n^{2ki} + \sum_{i=0}^{\frac{n}{2}-1} x_{2i+1} W_n^{2ki+k} \\ &= \sum_{i=0}^{\frac{n}{2}-1} x_{2i} W_{\frac{n}{2}}^{ki} + \left(\sum_{i=0}^{\frac{n}{2}-1} x_{2i+1} W_{\frac{n}{2}}^{ki} \right) \times W_n^k \end{aligned}$$

$$DFT(x, n)_k = DFT(\{x_0, x_2, \dots, x_{n-1}\}, \frac{n}{2})_k + DFT(\{x_1, x_3, \dots, x_{n-1}\}, \frac{n}{2})_k \times W_n^k, (k < \frac{n}{2})$$

该式可将长度为 n 的 DFT 分解为两个长度减半的子 DFT 问题递归求解。

上述拆分仅适用于 $k < \frac{n}{2}$ ，结合单位根对称性质 $W_{2n}^k = -W_{2n}^{k+n}$ ，可推导出后半区间计算公式：

$$DFT(x, n)_k = DFT(\{x_0, x_2, \dots, x_{n-1}\}, \frac{n}{2})_{k-\frac{n}{2}} - DFT(\{x_1, x_3, \dots, x_{n-1}\}, \frac{n}{2})_{k-\frac{n}{2}} \times W_n^{k+\frac{n}{2}}, (k \geq \frac{n}{2})$$

通过分治递归策略，FFT 将 DFT 计算复杂度优化至 $O(n \log n)$ ，逆序执行该流程即可实现点值表示向系数表示的逆向转换。

2.2 NTT 原理

标准 FFT 基于复数浮点运算实现，浮点乘法本身运算开销大、常数偏高，同时浮点精度误差会随运算层数累积，容易造成多项式乘法结果出错，不适用于密码学、同态加密这类要求精确整数运算的场景。

在整数多项式乘法场景中，无需依赖复数单位根，只需在有限模域内找到具备同等周期、正交性质的替代元即可。数论中的原根恰好满足这一要求，原根是模奇素数乘法群的生成元，而素数阶乘法群必为循环群，因此原根一定存在。

利用原根替代复数单位根，即可在整数模运算框架下实现与 FFT 等价的数论变换 NTT。但引入模乘运算后，常规取模操作计算开销同样偏大，因此引入 Montgomery 乘法对模乘过程进行化简加速。

递归版 NTT 依靠系统栈完成分治回溯，实际运行效率远低于迭代实现，同时递归开销会干扰 SIMD 优化效果的定量分析，因此本次实验全部采用迭代版 NTT 作为优化基准。

2.3 蝴蝶变换

递归 NTT 会按序列下标奇偶性拆分数据，分别求解子序列变换；而迭代版 NTT 不实时调整元素位置，必须在变换前或变换后完成下标重排，对应 FFT 的 DIT 与 DIF 两种经典实现流派。

蝴蝶变换严格数学推导流程繁琐，本文直接给出工程实现所用的递推公式：

$$r_i = \frac{r_{\frac{i}{2}} \mid \frac{imt}{2}}{2} \times (i \& 1)$$

3 SIMD 优化

3.1 NEON Intrinsics

SIMD 全称为单指令多数据并行技术，是处理器指令集的扩展架构，核心思想为单条指令同时对多组独立数据执行相同运算，大幅提升数据吞吐与计算并行度。

NEON 是 Arm Cortex-A 系列处理器标配的高级 SIMD 扩展指令集，也是本次 NTT 向量化优化的底层依赖。

实验中常用 NEON 向量寄存器类型如下：

Vectors

```
1 uint32x2_t, uint32x4_t, uint64x2_t
2 int32x2_t, int32x4_t
```

常用 NEON 内部函数如下：

Functions

```

1 vadd_u32, vsub_u32, vmul_u32 // 分别表示对 uint32x2_t 进行加法，减法和乘法
2 vaddq_u32, vsubq_u32, vmulq_u32 // 分别表示对 uint32x4_t 进行加法，减法和乘法
3 vmull_u32 // 表示将两个 uint32x2_t 相乘并存储在 uint64x2_t 中
4 vdup_n_u32 // 表示将一个 uint32_t 重复两次存储在一个 uint32x2_t 中
5 vdupq_n_u32 // 将一个 uint32_t 重复四次存储在一个 uint32x4_t 中
6 vcge_u32
// 将两个 uint32x2_t 进行对位比较，若第一个向量大于等于第二个向量，则置位全一，否则全0。
7 vset_lane_u32 // 改变 uint32x2_t 的单个通道的值
8 vget_lane_u32 // 获取 uint32x2_t 的单个通道的值

```

NEON 内置函数拥有规范命名体系：前缀标识运算类型，带 q 后缀为 128 位宽向量、无 q 为 64 位宽向量；含 lane 关键字代表单通道操作；后缀标识向量数据类型。

NEON 指令集未原生支持取模运算，若强行对向量各通道单独拆分解模，会引入大量分支与类型转换，反而抵消 SIMD 并行带来的性能收益，因此必须结合 Montgomery 乘法规避显式取模。

3.2 Montgomery 乘法

Montgomery 乘法是 NTT 实现 SIMD 向量化优化的必要基础，能够将高开销的模乘运算转化为移位、加法、乘法等简单整数运算。

3.2.1 Montgomery 乘法的底层原理：在取模乘法下对 2 的 n 次幂进行快速除法

Montgomery 算法核心依托模素数 p 下 2 的乘法逆元性质，满足 $\frac{p+1}{2}$ 为 2 在模 p 下的逆元，由此可得整除规则：

$$\frac{a}{2} \equiv \begin{cases} \frac{a}{2}, a \text{ is even} \\ \frac{a+p}{2}, a \text{ is odd} \end{cases} \pmod{p}$$

为求解 $x \cdot 2^{-k} \bmod p$ ，构造满足 $a \equiv x \pmod{p}$ 且 $a \equiv 0 \pmod{2^k}$ 的中间量 a ，直接对 a 右移 k 位即可等价完成模域下除以 2^k 。

构造式为 $a = x + p \cdot (x\mu)$ ，其中 $\mu \equiv -p^{-1} \pmod{2^k}$ 。预处理得到 μ 后，仅需两次乘法、一次加法与一次移位即可完成运算；计算 $x\mu$ 时对 2^k 取模，既能保证结果正确，又可规避数值溢出。

由约束 $p\mu < 2^k$ 可推得 $a < p + p \cdot 2^k$ ，进而 $\frac{a}{2^k} < 2p$ ，最终仅需至多一次条件减法即可完成结果规约，无需额外循环取模。

3.2.2 Montgomery 形式下的运算

定义模素数 p 下数值 x 的 Montgomery 表达为 $\bar{x} = x \cdot 2^k$ ，工程中 k 常取 32 或 64 位，利用无符号整数自然溢出特性实现快速模 2^k 。

Montgomery 域下加减法与普通模运算规则一致：

$$\bar{a} + \bar{b} \equiv \overline{a + b} \equiv (a + b) \cdot 2^k \pmod{p}$$

$$\bar{a} - \bar{b} \equiv \overline{a - b} \equiv (a - b) \cdot 2^k \pmod{p}$$

Montgomery 域乘法需额外补偿缩放因子，定义域内乘运算 $*$ ：

$$\bar{a} * \bar{b} \equiv \overline{a \cdot b} \equiv a \cdot b \cdot 2^k \cdot 2^{-k} \equiv \bar{a} \cdot \bar{b} \cdot 2^{-k} (\text{mod } p)$$

代入快速除法构造公式可得：

$$\bar{a} * \bar{b} \equiv \frac{\bar{a} \cdot \bar{b} + p \cdot \bar{a} \cdot \bar{b} \cdot \mu}{2^k} (\text{mod } p)$$

计算过程利用 32 位无符号数自然溢出自动对 2^k 取模。为保证规约结果满足 $\bar{a} * \bar{b} < 2p$ ，需满足 $2^k > p$ ，使运算后仅需一次减法即可完成范围校正。

整体一次模乘可拆解为 3 次乘法、1 次加法、1 次移位和至多 1 次减法，大幅简化取模逻辑。后文统一用 mod 代替素数 p ，适配代码开发规范。

3.3 使用 SIMD 优化

借助 Montgomery 乘法规避显式取模后，便可利用 NEON 向量指令并行加速乘法与蝶形运算。本次 SIMD 优化分为两大模块：基于向量指令实现 Montgomery 域四则运算、向量化重构 NTT 核心循环。

3.3.1 对于向量的选择

可选向量类型包含 `uint32x2_t`、`uint64x2_t`、`uint32x4_t` 三类，各有优劣：- `uint32x4_t`：单指令可并行处理 4 个数据，但乘法运算会溢出至 64 位，需要额外类型拆分，且仅当 NTT 步长 4 时才能发挥优势；- `uint64x2_t`：乘法无需类型转换，但高 32 位空间长期闲置，内存占用与指令开销偏高；- `uint32x2_t`：资源占用均衡、适配 NTT 各层步长迭代，实测性能表现最优。

综合测试对比后，最终选定 `uint32x2_t` 作为本次向量化优化的基础向量类型。

3.3.2 使用 SIMD 实现 Montgomery 形式下的运算

为适配多模数场景下的 NTT 测试，将 Montgomery 运算封装为独立类，每次切换模数自动重新计算相关参数，核心初始化代码如下：

Set New Modulus

```

1  class Montgomery32{
2  private:
3      void set_m_prime() {
4          uint32_t inv = 1;
5          for (int i = 0; i < 5; ++i) {
6              inv = inv * (2 - mod * inv);
7          }
8          m_prime = -inv;
9          m_prime_dup = vdup_n_u32(m_prime);
10     }
11     void set_r_mod() {
12         r_mod = (0x100000000ULL % mod); // 2^32 % mod
13     }
14     void set_r_squared_mod() {

```

```

15     r_squared_mod = ((uint64_t)r_mod * r_mod) % mod;
16     r_squared_mod_dup = vdup_n_u32(r_squared_mod);
17 }
18 public:
19     // 设置新模数
20     void set_mod(uint32_t new_mod) {
21         mod = new_mod;
22         modx2 = vdup_n_u32(mod);
23         set_m_prime();
24         set_r_mod();
25         set_r_squared_mod();
26     }
27 }

```

参数含义: r_mod 为 2^{32} 模 mod 结果; $r_squared_mod$ 为 $(2^{32})^2$ 模 mod 结果; m_prime 为 $\mu = -2^{-32} \pmod{mod}$, 通过牛顿迭代法快速求解。

基于 SIMD 实现 Montgomery 域模加、模减代码如下:

Add & Sub

```

1  uint32x2_t add(uint32x2_t a, uint32x2_t b) {
2      uint32x2_t sum = vadd_u32(a, b);
3      uint32x2_t mask = vcge_u32(sum, modx2);
4      return vsub_u32(sum, vand_u32(modx2, mask));
5  }
6
7  uint32x2_t sub(uint32x2_t a, uint32x2_t b) {
8      uint32x2_t diff = vsub_u32(a, b);
9      uint32x2_t mask = vclt_u32(a, b);
10     return vadd_u32(diff, vand_u32(modx2, mask));
11 }

```

利用 NEON 向量比较指令生成掩码, 无需分支判断即可完成模值规约, 避免串行分支带来的性能损耗。

向量化 Montgomery 乘法核心实现如下:

$$\bar{a} * \bar{b} \equiv \frac{\bar{a} \cdot \bar{b} + p \cdot \bar{a} \cdot \bar{b} \cdot \mu}{2^k} \pmod{p}$$

Montgomery Multiplication

```

1  uint32x2_t multiply(uint32x2_t a, uint32x2_t b) {
2      uint64x2_t T = vmull_u32(a, b);
3      uint32x2_t T_lo = vmovn_u64(T);
4      uint32x2_t m = vmul_u32(T_lo, m_prime_dup);
5      uint64x2_t mN = vmull_u32(m, modx2);
6      uint64x2_t sum = vaddq_u64(T, mN);
7      uint32x2_t res = vshrn_n_u64(sum, 32);
8      return vsub_u32(res, vand_u32(modx2, vcge_u32(res, modx2)));

```

9 }

该函数输入输出均为 Montgomery 域向量，依靠向量乘、加、移位与掩码规约完成完整模乘逻辑。普通整数与 Montgomery 域的相互转换可直接复用乘法接口实现。

3.3.3 使用 SIMD 改进 NTT

迭代版 NTT 包含三层循环结构：外层遍历蝶形步长、中层按步长分块、内层执行块内蝶形计算。当分块规模不小于 2 时，可通过 `uint32x2_t` 同时并行处理两组蝶形运算；步长小于 2 的边界情况采用标量串行处理。

SIMD 优化后 NTT 核心代码如下：

NTT with SIMD

```

1 void NTT(int op, int *F, uint32x2_t *G, int lmt, int mod){
2     //原根和原根求逆，用费马小定理计算
3     int g = 3, gi = qp(3, mod - 2, mod);
4     //蝴蝶变换
5     for(int i = 0; i < lmt; ++i)
6         if(i < r[i]) std::swap(F[i], F[r[i]]);
7     //步长小于2时无法向量化，直接计算
8     for(int i = 0; i < lmt; i += 2) {
9         int x = F[i], y = F[i + 1];
10        F[i] = x + y;
11        F[i] = F[i] >= mod ? F[i] - mod : F[i];
12        F[i + 1] = x - y;
13        F[i + 1] = F[i + 1] < 0 ? F[i + 1] + mod : F[i + 1];
14    }
15    //转换为Montgomery空间上的向量
16    for(int i = 0; i < lmt; i += 2)
17        G[i >> 1] = cal.to_mont((uint32x2_t){F[i], F[i + 1]});
18    for(int mid = 2; mid < lmt; mid <= 1) {
19        int wn = qp(op ? g : gi, (mod - 1) / (mid < 1), mod);
20        //每次走两步，需要将步长设置为2
21        uint32x2_t Wn = cal.to_mont(vdup_n_u32(1ll * wn * wn % mod));
22        for(int i = 0, R = mid < 1; i < lmt; i += R) {
23            uint32x2_t w = cal.to_mont((uint32x2_t){ 1, wn });
24            //每次走两步
25            for(int j = 0; j < mid; j += 2, w = cal.multiply(w, Wn)) {
26                uint32x2_t x = G[(i + j) >> 1], y = cal.multiply(w, G[(i + j + mid)
27                    >> 1]);
28                G[(i + j) >> 1] = cal.add(x, y);
29                G[(i + j + mid) >> 1] = cal.sub(x, y);
30            }
31        }
32    }

```

3.3.4 使用 SIMD 后的理论复杂度分析

从 SIMD 并行特性来看，向量化优化后单次 NTT 核心运算理论吞吐可提升一倍以上。但由于整体流程还包含蝴蝶变换、格式转换、数据拷贝等串行开销模块，受串行部分性能瓶颈限制，完整多项式乘法的实际加速比无法达到理论 2 倍。

3.4 使用 SIMD 优化后的性能分析

为量化评估优化效果，分别运行普通标量版 NTT 与 NEON SIMD 向量化版 NTT，借助 perf 工具采集运行耗时、指令数、时钟周期与 CPI 指标，缓存命中率对本次实验影响较小，不作重点分析。

实验测得运行耗时数据如表 1 所示：

Run Time(μs)	数据 1	数据 2	数据 3	数据 4
SIMD	0.03125	64.8726	63.9145	63.5821
Trivial	0.03214	83.6517	86.7234	90.1568

表 1: 运行时间

程序硬件性能指标如表 2 所示：

	Instructions	Circles	CPI
SIMD	2 156 892 453	1 048 726 315	0.47
Trivial	2 149 357 621	1 228 643 952	0.56

表 2: CPI

实测结果表明：SIMD 优化后 NTT 运算耗时由原先 88 微秒降至 65 微秒，性能提升明显；CPI 由 0.57 下降至 0.48。尽管向量化引入了格式转换、向量封装等额外指令，总指令数略有增加，但单周期指令执行效率大幅提升，时钟周期总数显著减少，最终实现整体运行耗时下降。

4 DIF & DIT

DIT 与 DIF 是 FFT 与 NTT 的两种经典实现范式，将 DFT 变换与蝴蝶变换视作线性矩阵操作，序列变换可表示为矩阵乘积形式：

$$DIT(x) = (butterfly \times DFT)(x)$$

$$DIF(x) = (DFT \times butterfly)(x)$$

矩阵乘法满足结合律，且蝴蝶变换矩阵为自逆矩阵。因此采用 DIF 架构实现正变换、DIT 架构实现逆变换，可在流程中完全省略独立蝴蝶变换步骤，既减少运算量，又规避标量与向量格式的频繁转换，进一步压缩运行开销。

DIF 与 DIT 分别从时域、频域角度推导得到，整体代码逻辑相近。引入该优化后的实测性能数据如下：

Run Time(μs)	数据 1	数据 2	数据 3	数据 4
SIMD	0.03125	64.8726	63.9145	63.5821
Trivial	0.03214	83.6517	86.7234	90.1568
DIF & DIT	0.34862	49.1532	48.8765	48.9217

表 3: 运行时间

	Instructions	Circles	CPI
SIMD	2 156 892 453	1 048 726 315	0.47
Trivial	2 149 357 621	1 228 643 952	0.56
DIF & DIT	2 125 647 189	952 368 741	0.44

表 4: CPI

5 总结

本次实验通过 NEON SIMD 指令集对迭代版 NTT 进行向量化并行优化，结合 Montgomery 乘法规避显式模运算瓶颈，有效降低了程序 CPI 与实际运行耗时。SIMD 优化后虽然因向量格式转换、Montgomery 域映射引入少量额外指令，但单指令多数据的并行特性大幅提升了硬件执行效率，时钟周期显著减少。

在此基础上引入 DIF/DIT 架构优化，取消独立蝴蝶变换步骤，进一步精简冗余运算。该优化对 CPI 改善幅度有限，主要通过减少总指令数、提升 SIMD 代码执行占比，进一步压缩时钟周期与运行时间。

本次 NTT 的两层优化实践，可为并行程序性能调优总结出三条通用思路：

1. 选用高效 SIMD 指令集与合理向量类型，直接降低程序 CPI 指标；
2. 调整代码结构，提升 SIMD 向量化模块在整体流程中的占比，间接优化平均 CPI；
3. 从算法层面精简冗余步骤、剔除无关运算，减少总执行指令数，从根源降低运行耗时。